Open Source Silicon Neuron: from schematics to GDS for Tiny Tapeout

Thomas Pigni

Institute of Neuroinformatics
Neuromorphic Cognitive Systems Group

MSc. Semester Project

Supervisors: Dr. Giacomo Indiveri
Jan 2026 – Apr 2026

Abstract

This report documents the continuation of the work carried out in the previous semester in *Open-Source Silicon Neuron: From Schematics to GDS*. The goal of the present project was to prepare a silicon neuron design for submission to the TinyTapeout IHP shuttle using an open-source analog design flow based on Xschem, NGSpice, KLayout, and the IHP SG13G2 open PDK. The final project files, including schematics and layouts are available in this github repository[1].

The initial objective was to implement a single adaptive silicon neuron with direct current input, without optimizing for area or power consumption. The main design priority was functional validation under the constraints of the TinyTapeout analog interface and the 1.2 V core supply. During the project, the circuit was progressively refined through schematic simulations, transistor sizing, current-mode characterization of the DPI block and integration with a voltage-to-current input stage.

As the tapeout deadline approached, the design strategy was adapted to reduce risk. In addition to the full adaptive neuron, a simplified leaky integrate-and-fire neuron with refractory behavior was developed as a more robust backup design. This simplification reduced the number of required bias pins and increased the probability of obtaining a layout-clean and LVS-clean submission within the available time.

The project also required substantial work on the open-source analog layout and verification flow. Several practical issues were addressed, including schematic-to-layout consistency, pMOS bulk connections, guard-ring representation in LVS, MIM capacitor placement, and discrepancies between schematic and extracted capacitor netlists. A complementary tutorial, *Open-Source Layout Workflow Using Xschem and KLayout*, was prepared to document the layout methodology and provide guidance for future users of the same flow.

[1] Repository link: https://github.com/sunny441/ttihp26a-adex-neuron-ncs

Contents

1 Circuit: Last Changes to Schematics

This section summarizes the final schematic-level modifications performed before starting the layout and verification phase. The original goal was to preserve the neuron architecture developed in the previous semester, while making the circuit compatible with the TinyTapeout IHP constraints, the available analog pins and the 1.2 V core supply. During this phase, several blocks were modified, simplified oor reorganized to increase the probability of obtaining a working and layout-clean design before the submission deadline. For reference, the final design from the previous semester is shown in Figure 1.

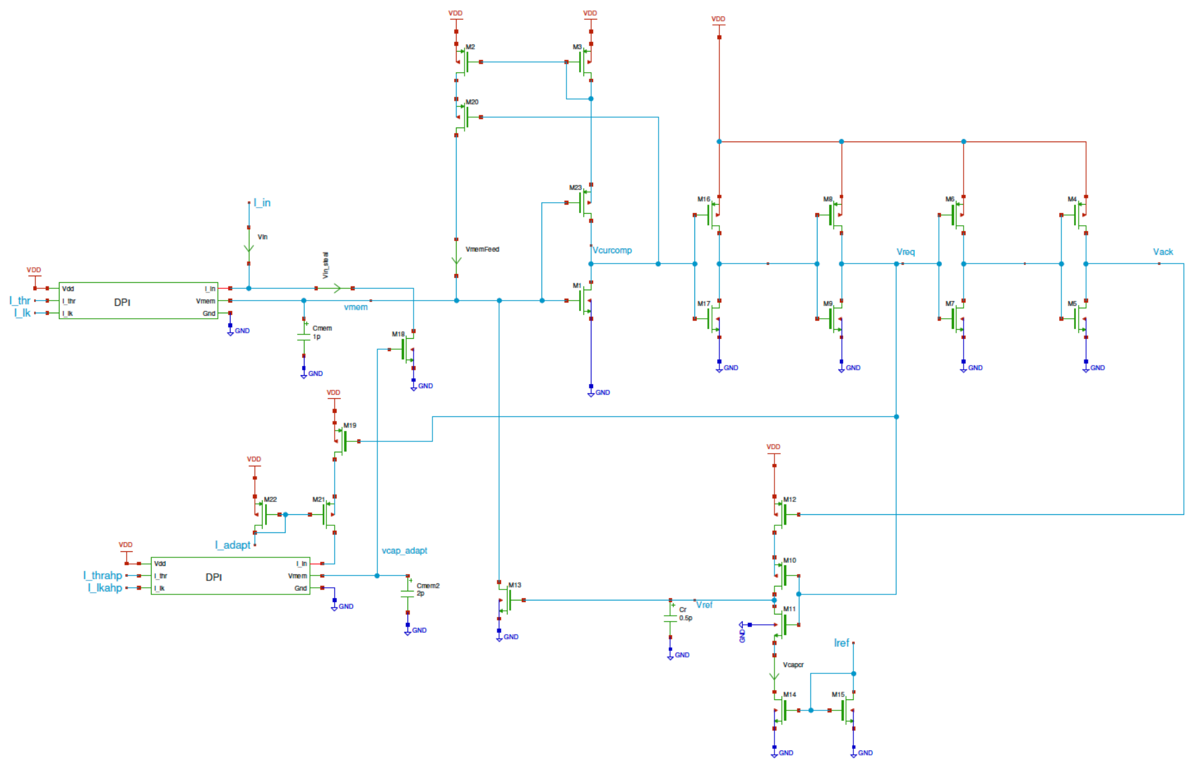

Figure 1: Circuit schematic of the neuron from the previous project.

1.1 Finger and Multiplier Interpretation

Before layout, the effect of transistor fingers and multipliers was checked in simulation. Different finger numbers and multiplier values were compared to verify their influence on device behavior. The simulations showed that changing the number of fingers had little effect on the simulated electrical behavior, as expected when the total device width remains unchanged. The results are shown in Figure 2 for different numbers of fingers and in Figure 3 for different multiplicities.

An important clarification was made regarding the interpretation of transistor parameters in Xschem. The width W shown in the schematic corresponds to the total width after finger subdivision, while the multiplier is applied on top of this width. Therefore, the effective device width is

$$W_{\text{eff}} = W \cdot m,$$

where m is the multiplier. This convention is consistent with the behavior normally used in commercial EDA tools.

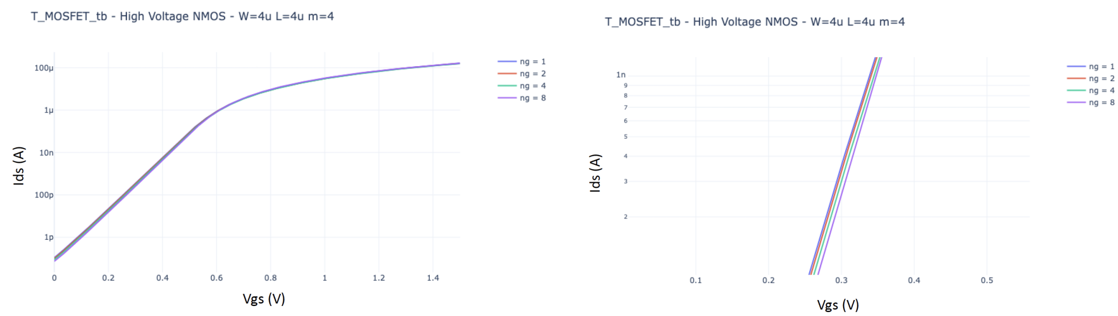

Figure 2: Simulation results for different numbers of fingers. The plot on the right shows a zoomed view.

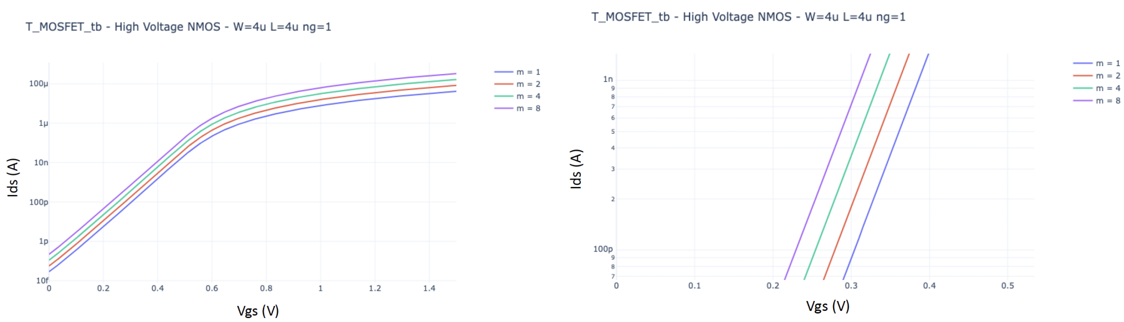

Figure 3: Simulation results for different multiplicities. The plot on the right shows a zoomed view.

1.2 DPI Block Refinement

The DPI block was kept as the central membrane integration circuit. The main objective was to obtain a current-mode low-pass response, where the membrane current follows an input current step with an approximately exponential rise and decay. Since the DPI is a current-mode circuit, the most relevant signal for characterization is the membrane current, rather than only the membrane voltage.

The DPI sizing was reviewed and modified several times. The channel lengths of the differential-pair transistors were increased from

$$L = 0.4 \text{ and } 1\ \mu\text{m} \quad \text{to} \quad L = 2\ \mu\text{m}$$

in order to reduce mismatch. This change was considered acceptable because the project was not strongly area-constrained at the single-neuron level. The priority was robustness rather than minimum area.

The sizing of the leak transistor, referred to as M_τ, was also modified. The goal was to obtain a more suitable relation between the leak current and the gain current. In particular, the circuit was adjusted so that the external analog bias voltage could generate the required leak current without using unnecessarily long or stacked devices. This also simplified the layout, because the updated devices could be drawn directly without splitting them into stacked transistor structures.

The final DPI schematic, including the transistor dimensions and the dummy transistors required to pass the LVS check, is shown in Figure 4.

1.3 Positive-Feedback Block Modification

The positive-feedback block was one of the most problematic parts of the schematic. In the previous version, the feedback current was too strong. Once the membrane voltage started to rise, the feedback current

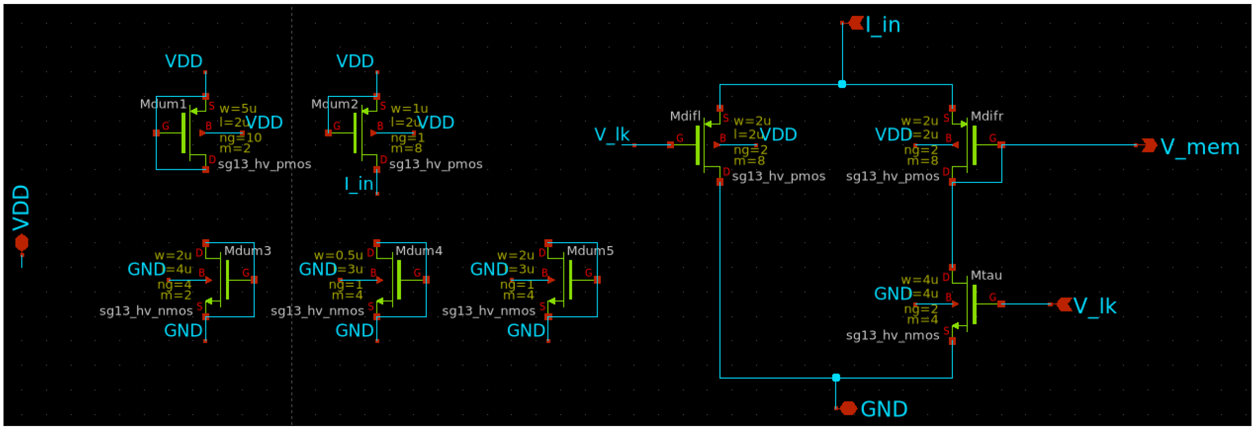

Figure 4: Final schematic of the DPI with dummy transistor for LVS on the left.

dominated the input current, causing the neuron dynamics to be controlled more by the feedback path than by the external input.

Figure 5 shows the effect of the positive-feedback block on the firing rate. The problematic version caused the firing rate to increase too aggressively, indicating that the feedback path was too strong. Therefore, the feedback strength had to be reduced so that the block would help trigger the spike transition without dominating the neuron dynamics.

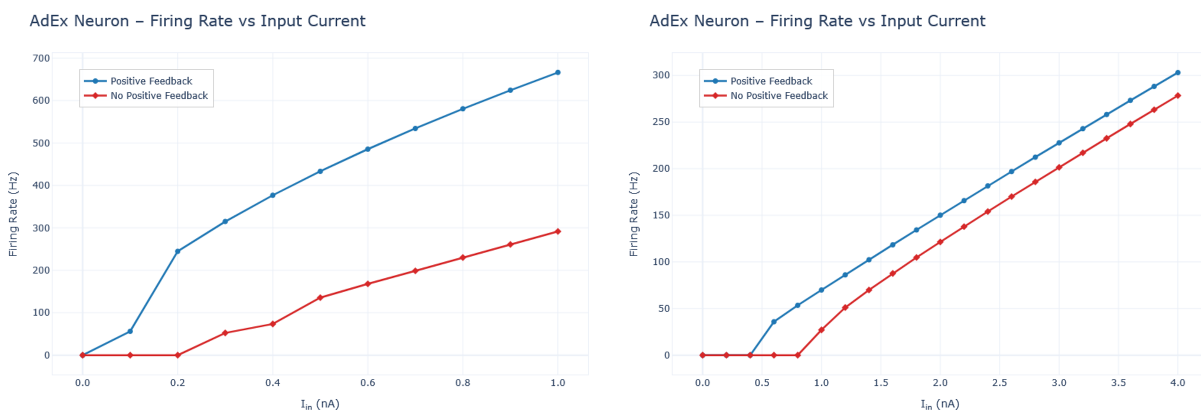

Figure 5: Effect of the positive-feedback block on the firing rate, without the refractory system. The comparison shows the behavior before and after reducing the feedback strength.

The original feedback mirror ratio was effectively too large. A much weaker feedback ratio was therefore investigated. Ratios such as $1:8$, $1:16$, and smaller effective feedback strengths were tested. The design principle adopted was that weak or almost negligible positive feedback is preferable to overly strong feedback. A dominant feedback path would make the neuron difficult to control and would compress the useful input-current range. The final schematic is shown in Figure 6.

1.4 Refractory System

The refractory system was kept as an essential part of the neuron. Its role is to reset or clamp the membrane node after a spike and impose a minimum inter-spike interval. This is important both for biological plausibility and for controlling the maximum firing frequency.

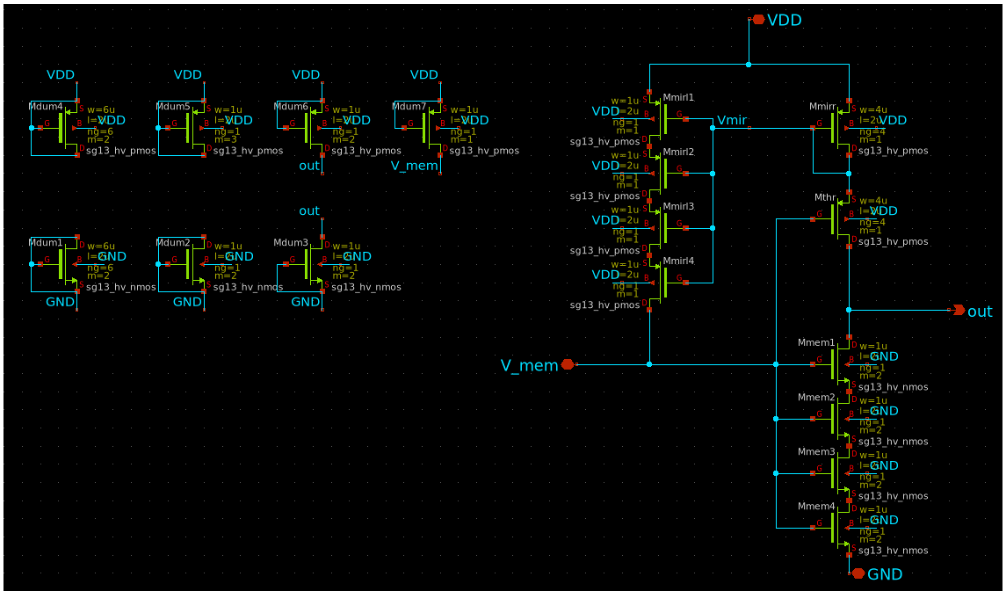

Figure 6: Final feedback-path schematic, including dummy transistors required to pass the LVS check.

During schematic refinement, the refractory period was adjusted so that the neuron would saturate around a target firing rate of approximately

$$f_{\text{sat}} \approx 300 \text{ Hz}.$$

This corresponds to a refractory period on the order of a few milliseconds. In later simulations, a refractory period of approximately

$$T_{\text{ref}} \approx 4 \text{ ms}$$

produced reasonable saturation behavior, as shown in Figure 7.

The refractory system was also used to define the digital handshaking behavior between the neuron and the external logic. In the final convention, the neuron generates the request signal, REQ, while the acknowledge signal, ACK, is provided by the external logic. The ACK signal then triggers the neuron reset through the refractory system. The final refractory-system schematic is shown in Figure 8.

1.5 Adaptation Block and Pulse-Width Issue

The original neuron included an adaptation block, implemented using an additional DPI-like circuit. This block was intended to integrate a spike-dependent adaptation current and reduce the firing rate after repeated spikes.

During testing, the adaptation block itself showed reasonable behavior. However, the spike request pulse driving it was too short, so a pulse extender was introduced.

The pulse extender improved the adaptation behavior, but it also introduced additional circuitry and required extra design effort. More importantly, the adaptation block required additional biasing and probing resources. Since the available analog pins were limited and the tapeout deadline was close, the adaptation block was eventually removed from this tapeout version.

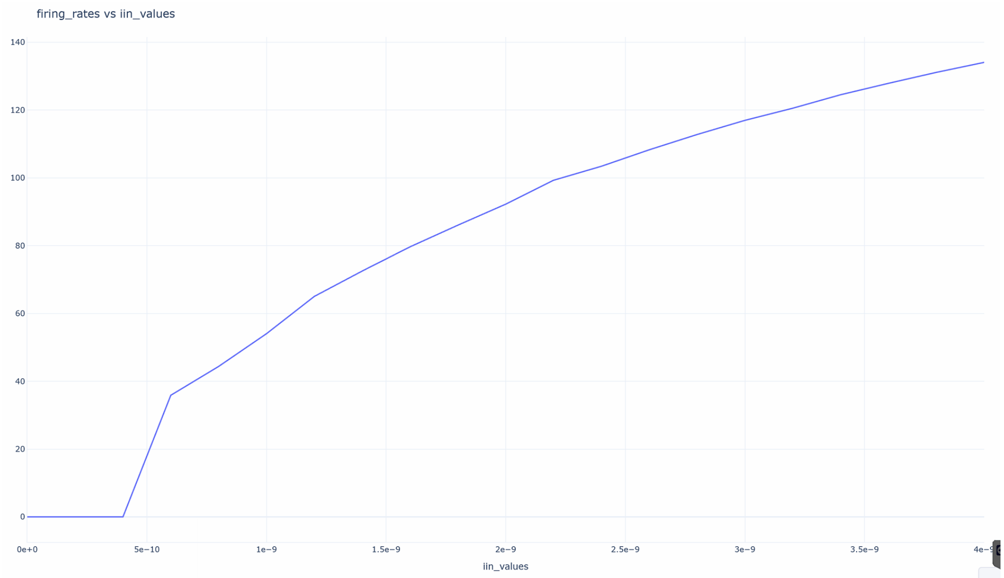

Figure 7: Saturation of the firing rate after adding the refractory system.

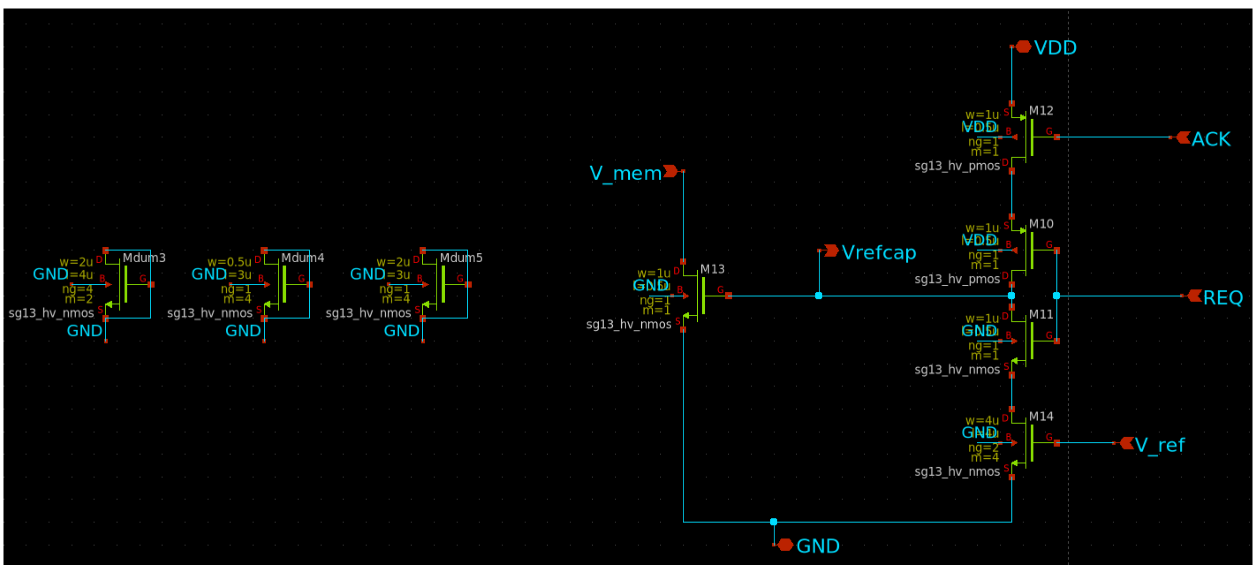

Figure 8: Final schematic of the refractory system.

1.6 Simplification to a LIF Neuron

To reduce the risk of missing the tapeout deadline a simplified neuron version was introduced. This version removes the adaptation block so it implements a leaky integrate-and-fire neuron with refractory behavior. The simplified architecture contains the following main blocks:

- membrane DPI.
- positive-feedback or thresholding block.
- refractory system.

These blocks are shown in Figure 9, which also shows the Xschem schematic block used for the tapeout.

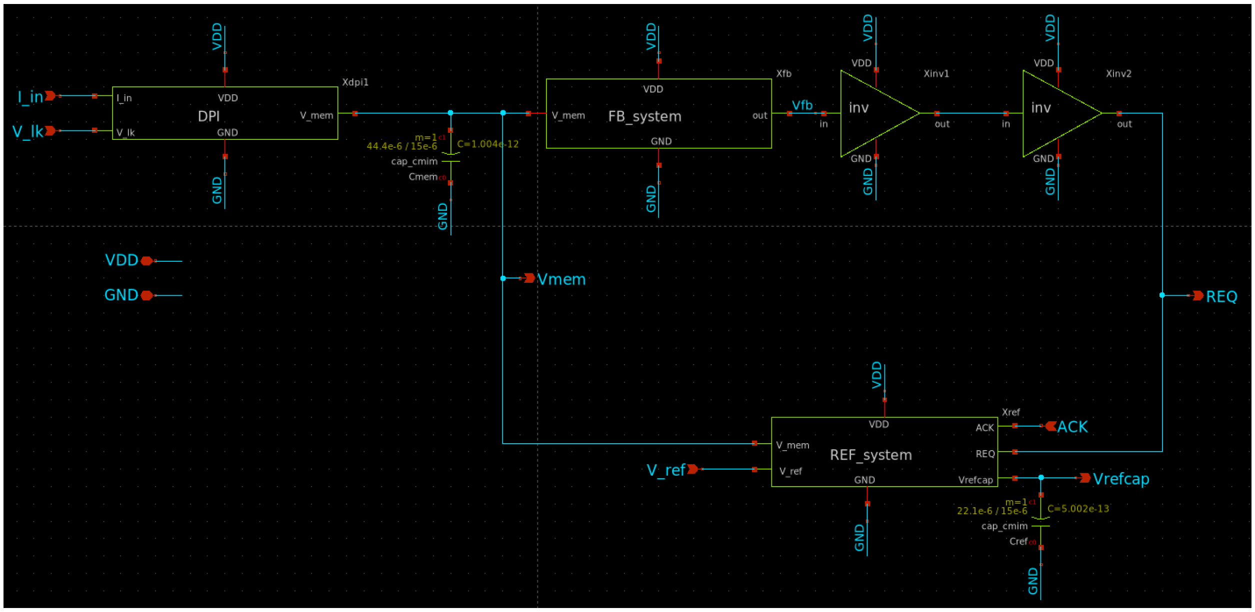

Figure 9: Xschem schematic block of the neuron used for the tapeout.

By removing the adaptation block, one analog pin became available for externally controlling the DPI gain. This was preferable to implementing an additional on-chip bias-generation block, especially given the limited time available before submission.

1.7 Neuron Interface Definition

The neuron interface was clarified during the final schematic updates. For the adaptive version, the relevant input and output signals were defined as:

- REQ: digital request output generated when the neuron spikes.
- ACK: digital acknowledge input provided externally.
- V_{mem}: analog membrane voltage output for probing.
- $V_{capAdapt}$: analog adaptation capacitor voltage output for probing.

For the simplified neuron without adaptation, the $V_{capAdapt}$ output is not required. However, V_{mem} remains important as an analog probe signal because it allows direct observation of the membrane dynamics.

It was also clarified that the delay or inverter chain between REQ and ACK should not be part of the neuron core itself. In the schematic testbench, such circuitry was used only to emulate the behavior of external digital logic. In the actual chip interface, ACK is treated as an input coming from the digital control system.

1.8 Instance and Device Renaming

The schematic hierarchy was cleaned up before layout. Generic instance names such as $X1$, $X2$, or $X5$ were replaced with more meaningful names. Examples include:

- $Xdpi$ for the membrane DPI block.

- Xfb for the feedback block.
- $Xres$ for the refractory system.

Individual transistors were also renamed where useful. For example, devices in the DPI were given names related to their function, such as M_τ for the leak transistor and names indicating the left and right branches of the differential pair.

This renaming was not only cosmetic. Clear naming made it easier to save internal simulation nodes, debug hierarchical signals, and compare schematic and layout netlists.

2 KLayout and Starting the Layout

After the final schematic modifications, the next step was to move from circuit simulation to full-custom layout. The layout work was performed using the open-source IHP SG13G2 flow, mainly based on KLayout for manual layout editing and verification. This phase required not only drawing the individual blocks, but also defining a common layout methodology so that several people could work in parallel and later integrate their cells into a single top-level design.

2.1 Initial Layout Strategy

At the beginning of the layout phase, the main objective was to obtain a manufacturable GDS as quickly as possible. Since this tapeout was intended primarily as a functional prototype, the layout was not optimized for minimum area or minimum power. The available area was considered sufficient for a single neuron; therefore, the layout rules and conventions were chosen to simplify routing, reduce DRC problems, and make LVS debugging easier.

The layout was split into smaller blocks, each corresponding to a schematic subcircuit. This allowed different team members to work in parallel on independent cells. Each block was laid out and verified separately before being integrated at the top level. This block-based approach was important because it made the debugging process more manageable. If a block was DRC- and LVS-clean individually, then later top-level problems could be isolated more easily to interconnects, pins, or hierarchy handling.

2.2 Difficulties Importing Schematics into KLayout

At the beginning, the intended workflow was to import the SPICE netlist generated by Xschem into KLayout and use it to assist with layout generation. However, this did not work reliably. Some symbols and subcircuits were not imported correctly, and in some cases transistors were missing, especially for digital or standard-cell-like parts of the schematic.

A possible alternative flow was therefore identified:

1. generate the SPICE netlist from Xschem.
2. import the circuit into Magic for device generation.
3. continue layout editing in KLayout.
4. export the final GDS.
5. import the GDS back into Magic or KLayout for LVS.
6. perform parasitic extraction.
7. simulate the extracted netlist.

In practice, once the correct LVS settings were understood, the work continued with manual instantiation of the SG13G2 transistors and KLayout-based DRC/LVS verification without the need to use magic to import the netlist. It should be noted that the importin netlist where you import in klayout the netlist of your circuit and klayout will automatically produce all the necessary transistors with already the correct transistor dimensions is still a problem and also the klayout dev say that is not possible for now.

Nevertheless, the initial import problems were important because they showed that the layout could not be treated as a fully automatic schematic-to-layout conversion. Manual device placement and careful schematic-layout consistency were required.

2.3 General Layout Rules Adopted by the Team

To make block integration easier, a common set of layout conventions was defined. These conventions were not necessarily minimum-area rules; instead, they were chosen to keep the design regular and reduce integration errors.

The top and bottom power rails of each cell were defined in Metal 3 with a width of

$$1\ \mu\text{m}.$$

The initial cell pitch used for several blocks was

$$20\ \mu\text{m},$$

measured from the center of one power rail to the center of the next power rail. An example is shown in Figure 10, where the pitch of the V2I cell is measured in KLayout.

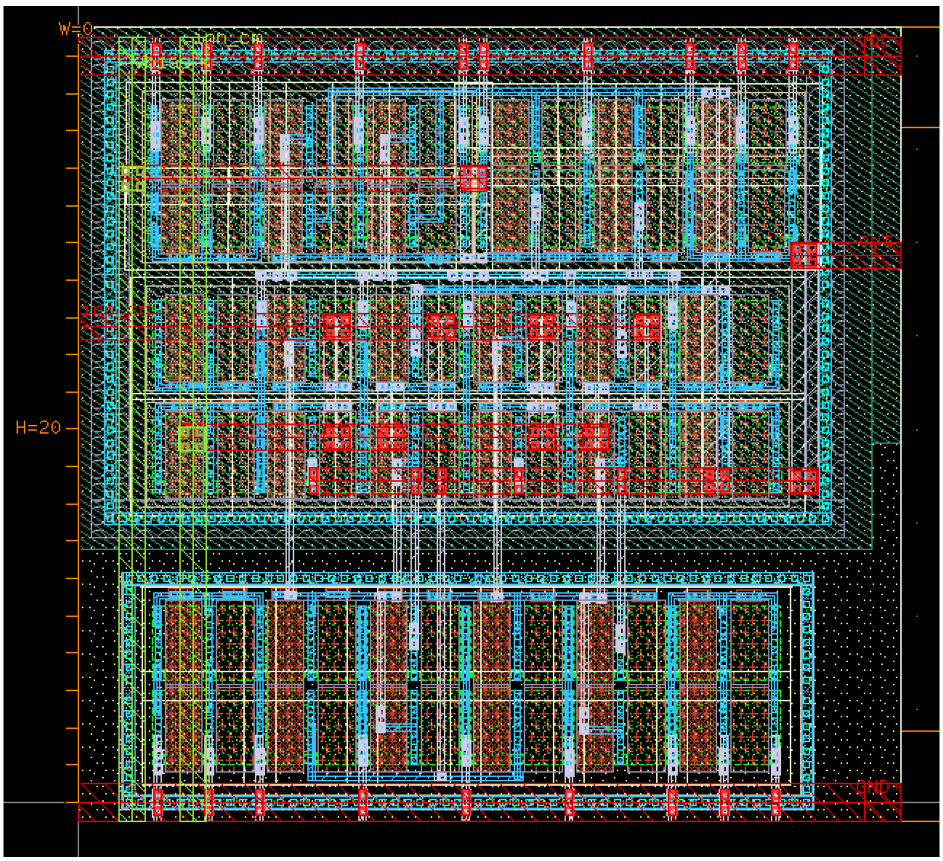

Figure 10: V2I cell pitch measured in KLayout. In red we can see metal 3 layers.

This pitch provided enough vertical space for transistor placement, local routing, and bias distribution. The routing direction convention was defined as follows:

- Metal 1 can be used in both horizontal and vertical directions.
- Metal 2 is preferably used for vertical routing.
- Metal 3 is preferably used for horizontal routing.

- Metal 4 is preferably used for vertical routing.
- Metal 5 is preferably used for horizontal routing.

For metals above Metal 1, direction changes should be made by switching layers through vias. This reduces routing ambiguity and makes top-level integration cleaner. An example is shown in Figure 11, where different metal layers are highlighted in the DPI layout.

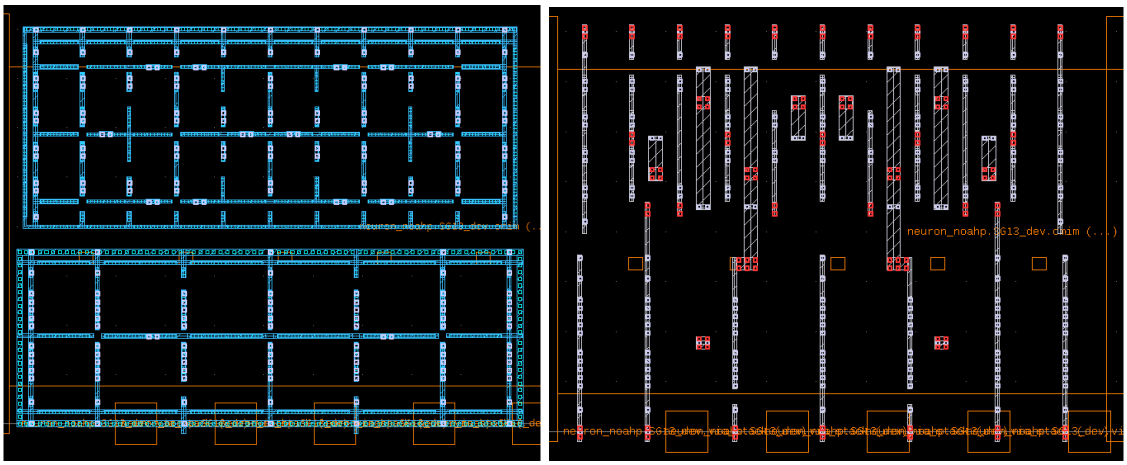

Figure 11: Highlighted routing layers in the DPI layout. The left image shows Metal 1, which is used omnidirectionally. The right image shows Metal 2, which is used only for vertical routing.

Bias signals such as V_{thr}, V_{lk}, and V_{ref} were planned to be routed vertically, mainly using Metal 4 and, where convenient, Metal 2. These vertical bias lines allow multiple neuron rows or repeated blocks to share common bias voltages. Local connections from these vertical bias lines to transistor gates were then made using lower metal layers.

2.4 Device Placement Conventions

The transistor placement strategy was also standardized. In each analog cell, pMOS devices were placed near the upper V_{DD} rail, while nMOS devices were placed near the lower V_{SS} rail. This follows the natural direction of current flow and simplifies bulk and substrate connections. An example is shown in Figure 12, where the inverter layout shows the pMOS and nMOS transistors placed close to their corresponding power rails.

All pMOS transistors inside a cell were placed in a common n-well whenever possible. This avoids unnecessary n-well spacing violations and reduces area. To make this possible, the pMOS bulk terminals were connected to V_{DD} in the schematic, instead of tying each bulk locally to its source. Local source-bulk connections would have required separate n-wells when source potentials differed, which was not desirable for this layout.

Similarly, p-substrate taps and n-well taps were included explicitly. These taps are important both electrically and for LVS, because the extracted layout contains substrate and well connections that must be represented consistently in the schematic.

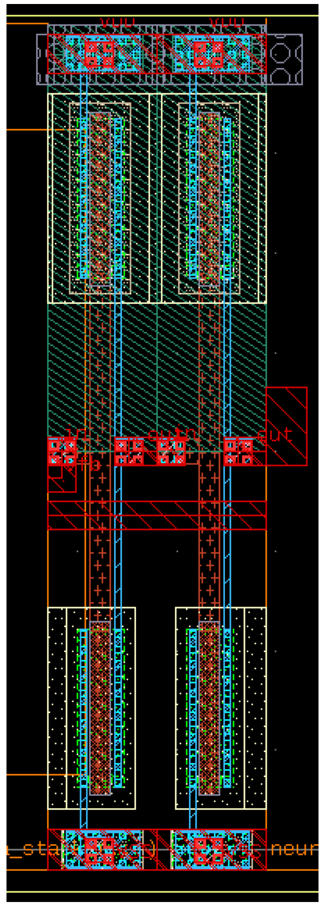

Figure 12: Inverter layout implementation showing the pMOS and nMOS transistors placed near the corresponding power rails. In the picture there are twon inverter connected in series.

2.5 Capacitor Layout Considerations

The membrane capacitor and other large capacitors were among the dominant area contributors, as shown in Figure 13. Their placement therefore had to be considered carefully. The relevant MIM capacitor density for the SG13G2 open PDK was taken as approximately

$$1.5\ \mathrm{fF}/\mu\mathrm{m}^2.$$

Placing the capacitors above the transistors would save horizontal area and keep the cell compact, while placing them in a separate row would simplify access to one terminal, especially when one plate is connected to ground.

The MIM capacitor uses upper metal layers, including TopMetal1. Therefore, it is not equivalent to placing a capacitor directly in Metal 1. It was expected that the capacitor could be placed above lower-level transistor routing, provided that no DRC or TinyTapeout-specific top-metal restriction was violated.

2.6 Use of KLayout DRC

KLayout was used from the beginning to run DRC checks on the manually drawn layouts. This was essential because several issues could be detected immediately at the block level, before top-level integration.

Typical early DRC problems included:

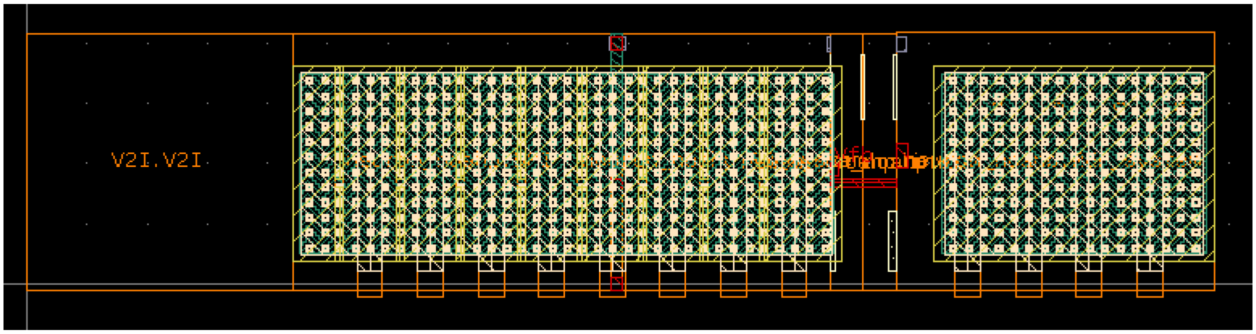

Figure 13: Full layout design with only the capacitors highlighted. The capacitors occupy a large fraction of the neuron circuit area.

- latch-up-related errors.
- missing or insufficient substrate taps.
- n-well spacing or connection problems.
- metal spacing and enclosure violations.

Running DRC frequently during layout was important. Instead of finishing a full block and only then checking it, it was better to progressively check intermediate versions. This reduced debugging time and made it easier to identify which recent modification introduced a violation.

At this point, the project moved from simply drawing layouts to systematically verifying each block with DRC and LVS. The next step was to ensure that the extracted layout netlists matched the Xschem schematics and that all blocks could be integrated into a final TinyTapeout-compatible top-level GDS.

3 Final Tapeout

The final tapeout phase consisted of integrating the verified blocks, resolving the remaining DRC and LVS issues, preparing the top-level GDS, and making the design compatible with the TinyTapeout checks. At this stage, the main objective was no longer to improve the neuron architecture, but to converge to a design that could be submitted reliably before the deadline.

For this reason, the final tapeout strategy was based on the simplified neuron version, namely a leaky integrate-and-fire neuron with refractory behavior.

3.1 Final Architecture

The final tapeout-oriented architecture was composed of the following main blocks:

- **V2I block**: converts an externally provided analog voltage into an input current for the neuron.
- **DPI membrane block**: integrates the input current and implements the leaky membrane dynamics.
- **feedback / threshold block**: detects the membrane threshold crossing and produces a spike-like transition.
- **refractory system**: resets or clamps the membrane node after a spike and limits the maximum firing rate.
- **digital interface**: provides the REQ output and receives the ACK input.
- **MUX and buffers**: interface the analog and digital neuron signals with the TinyTapeout infrastructure.

The complete adaptive version also included the adaptation DPI and the adaptation capacitor, but this version was considered secondary with respect to the simplified neuron. The simplified version was therefore used as the primary reference for layout integration and tapeout closure.

3.2 Top-Level Integration

The top-level layout integrated the neuron core with the V2I block, refractory system, feedback block, buffers, MUX, power rails, bias lines, and TinyTapeout interface pins. During this step, the most important task was to preserve consistency between the schematic hierarchy and the layout hierarchy.

The top-level schematic was updated to include the same subcircuits that were instantiated in the layout. Special care was required for:

- matching subcell names between Xschem and KLayout.
- using the same pin names in the schematic and layout.
- connecting power, ground, substrate, and n-well nets consistently.
- routing bias lines to all required blocks.
- exposing the selected analog probe signals.
- preserving the REQ and ACK digital interface convention.

The main analog probe signal kept in the final neuron was V_{mem}, because it allows the membrane integration dynamics to be observed directly after fabrication. In the adaptive version, the adaptation capacitor voltage $V_{capAdapt}$ was also considered useful for debugging, but it was not essential for the simplified tapeout version.

3.3 DRC Closure

Design rule checking was performed frequently throughout the final tapeout phase. The goal was to avoid accumulating many violations at the top level. Each block was checked individually before integration, and the complete design was checked again after assembly.

One important issue encountered during GDS preparation was the presence of zero-area polygons. These artifacts can be generated while moving or editing shapes in KLayout, or when generating GDS files from scripts. They can also appear after Boolean or difference operations. The zero-area polygon issue was particularly important because it could be missed by a normal local DRC run, but still be caught by the TinyTapeout CI checks.

The cleanup procedure in KLayout was:

1. open `Edit`
2. select `Search and Replace`
3. go to the `Delete` tab
4. select shapes
5. add the constraint `area = 0`
6. search for zero-area shapes
7. delete the zero-area boxes

This procedure was used to remove invalid polygons before running the final checks.

3.4 LVS Closure

Layout versus schematic verification was one of the most delicate parts of the final tapeout. The LVS flow required the Xschem-generated netlist and the KLayout-extracted layout netlist to represent the same hierarchy, devices, pins, and parasitic elements. This flow is described in more detail in the tutorial document. Several important LVS lessons were learned:

- guard rings and taps used in the layout must also appear in the schematic.
- p-substrate and n-well nets may be extracted as distinct nets and must therefore be handled explicitly.
- transistor bulk connections must match between schematic and layout.
- schematic device dimensions must match the dimensions used in layout.
- labels and pins must be named consistently across all hierarchy levels.

After these LVS-related changes, simulations had to be rerun because the schematic was no longer only an ideal schematic since it now also included the additional tap, well, substrate, or extracted elements required for layout consistency.

3.5 Final Design Status

By the end of the final tapeout phase, the project had reached the point where the main blocks were implemented and verified individually. The layout of one full custom LIF neuron is shown in Figure 14.

The final tapeout work was therefore not limited to producing one GDS file. A significant part of the effort was spent making the open-source analog design flow reliable enough for full-custom layout project. The final chip implements six neurons, which can be individually selected using a MUX. This allows the mismatch between different fabricated neurons to be measured. The final chip layout is shown in Figure 15.

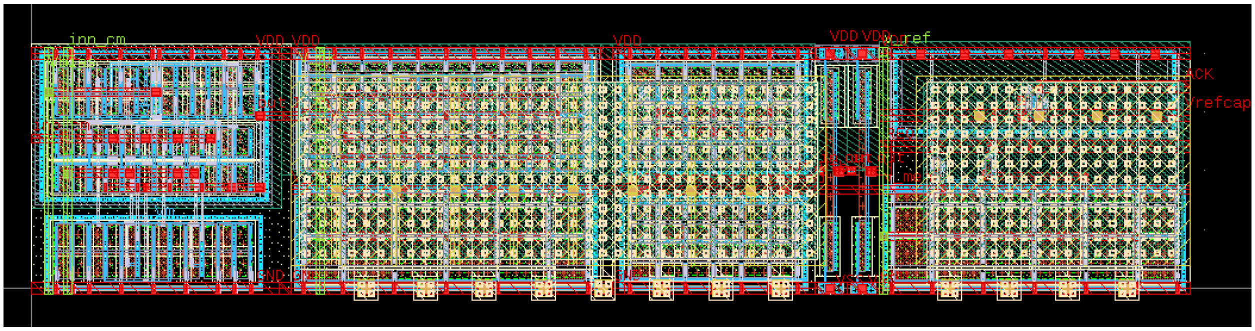

Figure 14: Full-custom neuron layout implementation.

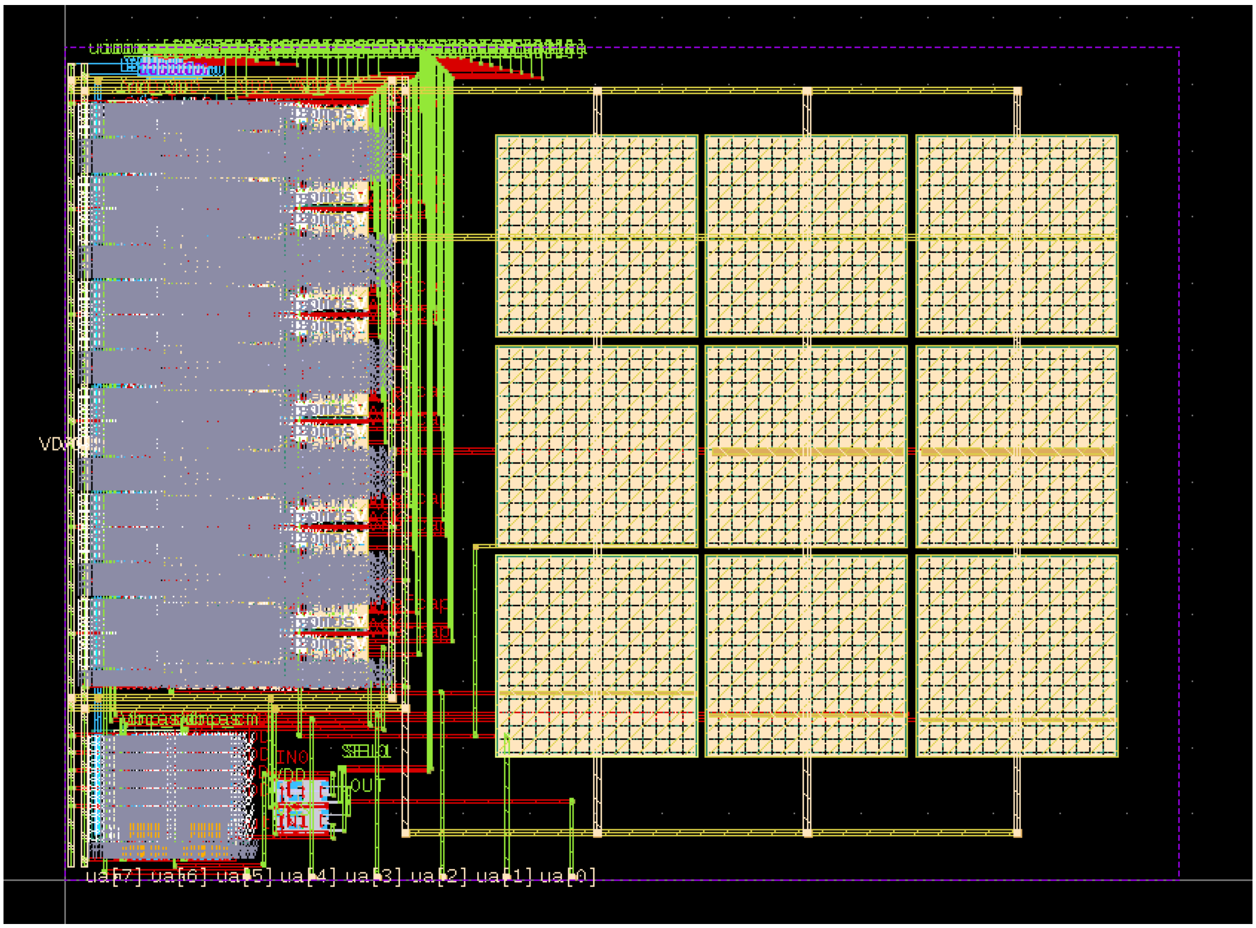

Figure 15: Top-level chip visualization. Due to the high density of interconnections, individual details are difficult to distinguish in this view.

The final result is a tapeout-oriented neuron design implemented using an open-source full-custom analog flow. Even though some architectural simplifications were made, the project achieved its main objective: moving from schematic-level neuron development toward a verified GDS suitable for fabrication on the TinyTapeout IHP shuttle.